SecureQT SDK (`secureqt-sdk`)

A Kotlin Multiplatform library for **end-to-end encrypted data transfer over a physical USB cable** using the **Android Open Accessory (AOA)** protocol.

The SDK ships as two binaries:

Artifact	Platform	Role	Use it in
`secureqt-sdk-jvm.jar`	JVM (Java 11+)	**USB Host / Initiator**	Desktop apps (Windows / Linux / macOS)
`secureqt-sdk-android.aar`	Android (minSdk 24)	**USB Accessory / Responder**	Android apps

- **Version:** `1.0.0` (`SecureQtSdk.VERSION`)
- **Package:** `com.example.secureqt.sdk`
- **Crypto:** ECDH P-256 ephemeral key exchange → AES-256-GCM for every payload

---

Table of Contents

1. How it works
2. Getting the binaries
3. Desktop (JAR) integration
4. Android (AAR) integration
5. Standard commands & wire contract
6. Custom commands
7. API reference
8. Limits, threading & gotchas
9. Troubleshooting

---

1. How it works

```
Desktop (JAR) — USB Host / Initiator          Android (AAR) — Accessory / Responder
──────────────────────────────────────        ──────────────────────────────────────

1. switchDeviceToAoa()
     AOA control requests 51/52/53 ─────────► device re-enumerates as accessory
                                              USB_ACCESSORY_ATTACHED intent fires
2. re-scan bus, open, claim interface
3. createDesktopTransport()                   createAndroidAccessoryTransport()
4. performHandshake(isInitiator = true) ◄─► performHandshake(isInitiator = false)
          (ECDH P-256 → shared AES-256-GCM key)
5. createTransferClient()
     sendFile / fetchFile / ... ──────────────► receiveSecure() → parse → handle
                                ◄────────────── reply via sendSecure()
```

Layers (all in `commonMain`, shared by both binaries):

Layer	Class	Responsibility
Transport	`IUsbTransport`	Raw byte read/write over USB

Layer	Class	Responsibility
Framing	`PacketCodec`, `PacketTypes`	`[1: type][4: length][payload]` packets
Security	`SecureChannel`	ECDH handshake, AES-GCM encrypt/decrypt, framing
Commands	`CommandPayloadBuilder` / `CommandPayloadParser`	`[1: commandId][args…]` payloads
High-level client	`ISecureTransferClient`	File/folder send, fetch, list, delete, rename… (host side)

> **Important:** the high-level `ISecureTransferClient` is the **host side only**. The Android side gets the transport + channel + parser. **Your Android app implements the command handlers** (see §4.5).

---

2. Getting the binaries

Option A: Download from GitHub Releases

Pushing a tag matching `sdk-v*` (for example `sdk-v1.0.0`) runs `.github/workflows/sdk-release.yml`, which attaches these files to the release:

- `secureqt-sdk-jvm.jar`
- `secureqt-sdk-android.aar`
- `usb-driver-installer-standalone.jar` (Windows WinUSB driver helper)

Option B: Build locally

From the repository root:

```
# JVM JAR with all dependencies bundled → secureqt-sdk/build/libs/secureqt-sdk-jvm-1.0.0-all.jar
# (this is the file released as secureqt-sdk-jvm.jar)
./gradlew :secureqt-sdk:jvmStandaloneJar

# Android AAR → secureqt-sdk/build/outputs/aar/secureqt-sdk-release.aar
./gradlew :secureqt-sdk:bundleReleaseAar
```

Requires JDK 17 to run Gradle (the SDK compiles to JVM 11 bytecode) and Android SDK platform 36.

Option C: Same Gradle build (module dependency)

```
// settings.gradle.kts
include(":secureqt-sdk")

// consumer build.gradle.kts
dependencies { implementation(project(":secureqt-sdk")) }
```

---

3. Desktop (JAR) integration

3.1 Add dependencies

`secureqt-sdk-jvm.jar` is **self-contained**. It bundles everything it needs at runtime, so it's the only dependency you add:

- `usb4java` 1.3.0, with `libusb4java` natives for Windows (x86, x86-64), Linux (x86, x86-64, ARM, AArch64) and macOS (x86-64)
- `commons-lang3` 3.8.1 (required by usb4java)
- `kotlinx-coroutines-core` 1.10.2
- `kotlin-stdlib` 2.3.20

```
// build.gradle.kts (Desktop / JVM project)
dependencies {
    implementation(files("libs/secureqt-sdk-jvm.jar"))
}
```

Maven

```
<dependency>
  <groupId>com.example.secureqt</groupId>
  <artifactId>secureqt-sdk-jvm</artifactId>
  <version>1.0.0</version>
  <scope>system</scope>
  <systemPath>${project.basedir}/libs/secureqt-sdk-jvm.jar</systemPath>
</dependency>
```

> **Version conflicts:** the bundled libraries aren't relocated, because the SDK's public API uses usb4java and Kotlin types directly. If your app also declares Kotlin, coroutines, usb4java or commons-lang3, the first copy on the classpath wins. Use versions at least as new as those listed above, and put your own dependencies first.

3.2 OS prerequisites

OS	Requirement
Windows	The AOA device (VID `0x18D1`, PID `0x2D00` / `0x2D01`) needs a **WinUSB** driver, and `adb` must not hold the device. See §3.7.
Linux	A udev rule so non-root users can open the phone and the AOA device. See §3.8.
macOS	Intel only: works with the bundled libusb from usb4java; no driver needed. Apple Silicon needs an x86-64 JDK under Rosetta.

3.3 Minimal host flow (quick start)

The example below shows the bare sequence. For a production host that handles device polling and the Windows and Linux differences (driver binding, `adb`, retries), use the built-in session manager from §3.5.

```
import com.example.secureqt.sdk.SecureQtSdk
import com.example.secureqt.sdk.createDesktopTransport
import com.example.secureqt.sdk.findDesktopBulkEndpoints
import com.example.secureqt.sdk.switchDeviceToAoa
import kotlinx.coroutines.runBlocking
import org.usb4java.*
```

```
import java.io.File

const val AOA_VID: Short = 0x18D1
val AOA_PIDS = 0x2D00..0x2D05

fun findDevice(predicate: (DeviceDescriptor) -> Boolean): Device? {
    val list = DeviceList()
    LibUsb.getDeviceList(null, list)
    try {
        for (dev in list) {
            val desc = DeviceDescriptor()
            LibUsb.getDeviceDescriptor(dev, desc)
            if (predicate(desc)) return LibUsb.refDevice(dev)
        }
    } finally {
        LibUsb.freeDeviceList(list, true)
    }
    return null
}

fun main() = runBlocking {
    check(LibUsb.init(null) == LibUsb.SUCCESS)

    // ── 1. Find the phone and switch it into AOA mode (skip if already AOA) ──
    var aoa = findDevice { it.idVendor() == AOA_VID && (it.idProduct().toInt() and 0xFFFF) in AOA_PIDS }
    if (aoa == null) {
        val phone = findDevice { it.idVendor() == 0x04E8.toShort() } // your phone's normal VID
            ?: error("No Android device found")
        // manufacturer/model MUST match the Android app's accessory_filter.xml
        SecureQtSdk.switchDeviceToAoa(phone, manufacturer = "AndroidOpenAccessory", model = "DataTransfer
")

        // ── 2. Device disconnects and re-enumerates as 18D1:2D0x; poll for it ──
        repeat(20) {
            Thread.sleep(500)
            aoa = findDevice { it.idVendor() == AOA_VID && (it.idProduct().toInt() and 0xFFFF) in AOA_PID
S }
            if (aoa != null) return@repeat
        }
    }
    val device = aoa ?: error("Device did not re-enumerate in AOA mode")

    // ── 3. Open + claim the interface (the SDK does NOT claim it for you) ──
    val handle = DeviceHandle()
    check(LibUsb.open(device, handle) == LibUsb.SUCCESS)
    val ep = SecureQtSdk.findDesktopBulkEndpoints(device) ?: error("No bulk endpoints")
    LibUsb.setAutoDetachKernelDriver(handle, true)
    check(LibUsb.claimInterface(handle, ep.interfaceNumber) == LibUsb.SUCCESS)

    val transport = SecureQtSdk.createDesktopTransport(
        handle = handle,
        inEp = ep.inAddr,
        outEp = ep.outAddr,
        ifaceNum = ep.interfaceNumber
    )

    // ── 4. Encrypted channel + ECDH handshake (host is the initiator) ──
    val channel = SecureQtSdk.createChannel(transport)
    if (!channel.performHandshake(isInitiator = true)) error("Handshake failed")

    // ── 5. High-level operations (every function: §3.10; sending a .map file: §3.12) ──
    val client = SecureQtSdk.createTransferClient(channel)

    client.listRemoteFiles("/sdcard/Download").forEach {
        println("${if (it.isDirectory) "[D]" else "    "} ${it.name}  ${SecureQtSdk.Utils.formatSize(it.si
```

```
ze)}")
    }

    client.sendFile(File("photo.jpg"), "/sdcard/Download") { sent, total ->
        println("Upload ${sent * 100 / total}%")
    }

    client.fetchFile("/sdcard/Download/report.pdf", File("downloads"))
    client.sendDirectory(File("myFolder"), "/sdcard/Download")     // streamed as ZIP
    client.fetchRemoteDirectory("/sdcard/DCIM/Camera", File("backup"))
    client.createFolder("/sdcard/Download/NewFolder")
    client.renameFile("/sdcard/Download/a.txt", "b.txt")
    client.deleteFile("/sdcard/Download/b.txt")

    // ── 6. Tear down ──
    client.sendDisconnect()
    transport.close()   // releases interface, re-attaches kernel driver, closes handle
    LibUsb.unrefDevice(device)
    LibUsb.exit(null)
}
```

3.4 Cancelling a transfer

```
val job = scope.launch { client.sendFile(bigFile, "/sdcard/Download") }
// later, e.g. from a "Cancel" button:
client.cancelTransfer()   // sends TYPE_CANCEL; the in-flight call returns false
```

3.5 Production host for Windows & Linux

Recommended: the built-in session manager

`SecureQtSdk.createDesktopSessionManager()` handles device polling, plug and unplug detection, the AOA switch, re-enumeration, the handshake and native handle cleanup:

```
import com.example.secureqt.sdk.SecureQtSdk
import com.example.secureqt.sdk.createDesktopSessionManager
import com.example.secureqt.sdk.host.DesktopUsbDeviceManager

val deviceManager = DesktopUsbDeviceManager()
val sessions = SecureQtSdk.createDesktopSessionManager(deviceManager)
sessions.startPolling()

// Show sessions.sessionsState in your UI, then connect the device the user picks.
// Don't connect to every device: detection matches vendor IDs, so non-phone devices can appear.
scope.launch {
    sessions.getSession(deviceId)?.connect()   // switches to AOA and performs the handshake
}

// Once a session's status is Ready
sessions.getSession(deviceId)?.client?.sendFile(File("photo.jpg"), "/sdcard/Download")

// On shutdown
sessions.close()
deviceManager.cleanup()
```

- Several phones can connect at once. Each gets its own session, encryption keys and `client`, and a session only ever connects to the phone on its own USB port. Transfers on different clients run in parallel.

- Devices already in AOA mode connect automatically. Other devices wait for `connect()`, so you decide when a phone gets switched.
- A failed connection moves to `DesktopSessionStatus.Error` and is not retried until you call `connect()` again.
- To handle missing Windows drivers, implement `UsbDriverSupport` and pass it to `DesktopUsbDeviceManager(driverSupport = ...)`.
- To pass diagnostics to your own logging, pass an `ILogger` to both constructors. The default writes to standard output.
- On Windows the connection stops `adb` if it may be holding the device, which affects the whole machine. To prevent that, use your own session factory with `DesktopUsbConnection(killAdbOnWindows = false)`.
- To add per-device state such as a transfer queue, implement `ManagedUsbSession` and pass your session type to `DesktopUsbSessionManager`.

The rest of this section shows how to build a host layer by hand, for when you need different retry or driver behaviour.

The SDK gives you the AOA switch, endpoint discovery and the transport. Making the connection reliable on a real PC needs OS-specific handling, which is what this host layer adds:

Concern	Windows	Linux
USB access	libusb needs a **WinUSB** (or libusbK) driver bound to the device	libusb uses `usbfs`; the user needs **read/write permission** on `/dev/bus/usb/BBB/DDD` (udev rule)
Opening the phone **before** the AOA switch	Needs at least one interface with a WinUSB-compatible driver (for example the ADB interface with USB debugging on). Otherwise install WinUSB for the phone's own VID:PID	Works for any user the udev rule allows
Opening the AOA device (`18D1:2D00/2D01`)	Needs WinUSB on that VID:PID. On `2D01` (AOA + ADB composite) it must be on interface 0 (`MI_00`)	Works once permissions apply
Typical `LibUsb.open` errors	`-3 ERROR_ACCESS` (`adb.exe` holds the device), `-12 ERROR_NOT_SUPPORTED` (no WinUSB, or Plug and Play binding not finished)	`-3 ERROR_ACCESS` (no udev rule, or udev hasn't applied it yet just after re-enumeration)
Kernel driver on the interface	Not applicable (`setAutoDetachKernelDriver` isn't supported)	Detach it before claiming (`setAutoDetachKernelDriver(true)`)
Time to re-appear after the AOA switch	1–3 s, plus driver binding	~0.5–1.5 s
Admin rights	Only for the one-time driver install (UAC prompt)	Only for the one-time udev rule install

`HostPlatform`: per-OS behaviour

```
package com.yourcompany.secureqt.host

import com.example.usbdriverinstaller.api.IUsbDriverInstaller
import com.example.usbdriverinstaller.api.UsbDriverInstallerFactory
import org.usb4java.LibUsb
import java.io.File
import java.util.concurrent.TimeUnit

/** OS-specific policy used by [SecureQtHost]. */
```

```
interface HostPlatform {
    /** Extra wait after the AOA device appears, so the OS can finish binding drivers. */
    val settleDelayMs: Long

    /** Called once at the start of connect(), before the AOA switch. */
    fun prepare() {}

    /**
     * Called when `switchDeviceToAoa` fails on the phone in normal mode.
     * @return true to retry the switch.
     */
    fun onSwitchFailed(vid: Int, pid: Int, attempt: Int): Boolean = false

    /**
     * Called when opening/claiming the AOA device fails with libusb [error].
     * @return true to retry (the host re-scans the bus before each retry).
     */
    fun onOpenFailed(vid: Int, pid: Int, error: Int, attempt: Int): Boolean

    companion object {
        fun forCurrentOs(): HostPlatform {
            val os = System.getProperty("os.name", "").lowercase()
            return when {
                "win" in os -> WindowsHostPlatform()
                "linux" in os -> LinuxHostPlatform()
                else -> DefaultHostPlatform()     // macOS: no driver or permission work needed
            }
        }
    }
}

class DefaultHostPlatform : HostPlatform {
    override val settleDelayMs = 300L
    override fun onOpenFailed(vid: Int, pid: Int, error: Int, attempt: Int) = attempt < 5
}
```

Windows implementation

```
class WindowsHostPlatform(
    /** WinUSB installer from `usb-driver-installer-standalone.jar`; null = never install drivers. */
    private val installer: IUsbDriverInstaller? =
        runCatching { UsbDriverInstallerFactory.getDefaultInstaller() }.getOrNull(),
    /** Set false if your users must keep MTP (Explorer access) on the phone's normal VID:PID. */
    private val allowDriverInstallOnPhone: Boolean = true
) : HostPlatform {

    override val settleDelayMs = 800L          // Plug and Play needs time to bind the new device node

    /** adb.exe opens every Android device it sees and blocks libusb with ERROR_ACCESS. */
    override fun prepare() = killAdbServer()

    override fun onSwitchFailed(vid: Int, pid: Int, attempt: Int): Boolean {
        // The phone has no WinUSB-compatible interface (USB debugging off, or no ADB driver).
        if (attempt > 1 || !allowDriverInstallOnPhone) return false
        return installWinUsb(vid, pid, "Android Device")
    }

    override fun onOpenFailed(vid: Int, pid: Int, error: Int, attempt: Int): Boolean = when (error) {
        LibUsb.ERROR_ACCESS, LibUsb.ERROR_BUSY -> {
            killAdbServer()
            attempt < 16
        }
        LibUsb.ERROR_NOT_SUPPORTED, LibUsb.ERROR_NOT_FOUND -> {
```

```
            // Retry the first few times: the driver may be installed but not bound yet.
            // Only after that, install WinUSB for the AOA VID:PID.
            if (attempt == 4) installWinUsb(vid, pid, "Android Open Accessory") else attempt < 16
        }
        else -> attempt < 3
    }

    private fun installWinUsb(vid: Int, pid: Int, name: String): Boolean {
        val inst = installer ?: return false
        if (inst.isDriverAlreadyInstalled(vid, pid)) return true
        // One UAC prompt. installWinUsb() also removes conflicting OEM drivers and
        // rescans/restarts the device, so don't call cleanupConflictingDrivers()/
rescanAndRestartDevice() too.
        val result = inst.installWinUsb(vid, pid, name)
        if (!result.isSuccess) {
            System.err.println("[SecureQT] WinUSB install failed for %04X:%04X: $
{result.message}".format(vid, pid))
            return false
        }
        Thread.sleep(1500)                                          // let Plug and Play bind the driver
        return true
    }

    private fun killAdbServer() {
        runCatching {
            val sdkAdb = System.getenv("LOCALAPPDATA")
                ?.let { File(it, "Android\\Sdk\\platform-tools\\adb.exe") }
                ?.takeIf { it.exists() }
            val cmd = if (sdkAdb != null) listOf(sdkAdb.absolutePath, "kill-server")
                      else listOf("taskkill", "/F", "/IM", "adb.exe")
            ProcessBuilder(cmd).redirectErrorStream(true).start().waitFor(3, TimeUnit.SECONDS)
        }
    }
}
```

Linux implementation

```
class LinuxHostPlatform : HostPlatform {

    override val settleDelayMs = 300L

    override fun onOpenFailed(vid: Int, pid: Int, error: Int, attempt: Int): Boolean {
        // Right after re-enumeration, udev may not have applied the rule yet, so retry briefly.
        if (attempt < 6) return true
        if (error == LibUsb.ERROR_ACCESS) {
            System.err.println(
                "[SecureQT] Permission denied on %04x:%04x. Install the udev rule (README §3.8),
".format(vid, pid) +
                "then unplug and replug the phone."
            )
        }
        return false
    }
}
```

`SecureQtHost`: the cross-platform host

```
package com.yourcompany.secureqt.host

import com.example.secureqt.sdk.SecureQtSdk
import com.example.secureqt.sdk.client.ISecureTransferClient
```

```
import com.example.secureqt.sdk.createDesktopTransport
import com.example.secureqt.sdk.findDesktopBulkEndpoints
import com.example.secureqt.sdk.switchDeviceToAoa
import com.example.secureqt.sdk.transport.desktop.DesktopAoaTransport
import org.usb4java.Context
import org.usb4java.Device
import org.usb4java.DeviceDescriptor
import org.usb4java.DeviceHandle
import org.usb4java.DeviceList
import org.usb4java.LibUsb

/**
 * Finds an Android phone, switches it to AOA mode, opens it with OS-specific recovery,
 * and performs the SecureQT handshake. One instance = one phone connection.
 */
class SecureQtHost(
    private val platform: HostPlatform = HostPlatform.forCurrentOs(),
    /** Must match the Android app's res/xml/accessory_filter.xml. */
    private val manufacturer: String = "AndroidOpenAccessory",
    private val model: String = "DataTransfer",
    /** Restrict to one phone, e.g. 0x04E8 (Samsung). null = any known Android vendor. */
    private val targetVid: Int? = null
) : AutoCloseable {

    companion object {
        const val AOA_VID = 0x18D1
        val AOA_PIDS = 0x2D00..0x2D05
        val ANDROID_VIDS = setOf(
            0x18D1, 0x04E8, 0x2717, 0x2A70, 0x22D9, 0x2B4C, 0x1EBF, 0x22B8, 0x0FCE, 0x1004,
            0x0BB4, 0x19D2, 0x05C6, 0x0E8D, 0x12D1, 0x0B05, 0x17EF, 0x29D1, 0x3170, 0x1949
        )
        private const val MAX_OPEN_ATTEMPTS = 20
    }

    private val ctx = Context()
    private var transport: DesktopAoaTransport? = null
    var client: ISecureTransferClient? = null
        private set

    init {
        val r = LibUsb.init(ctx)
        check(r == LibUsb.SUCCESS) { "LibUsb.init failed: ${LibUsb.strError(r)}" }
    }

    /** Full connect: switch → re-enumerate → open/claim → ECDH handshake. */
    suspend fun connect(): ISecureTransferClient {
        platform.prepare()

        // 1. Switch to AOA unless the phone is already in accessory mode
        if (findDevice(isAoa) == null) {
            val phone = findDevice { vid, pid -> !isAoa(vid, pid) && (targetVid?.let { it == vid } ?: (vi
d in ANDROID_VIDS)) }
                ?: error("No Android device found. Check the cable and that the phone is unlocked.")
            try {
                val (vid, pid) = ids(phone)
                var attempt = 1
                while (!SecureQtSdk.switchDeviceToAoa(phone, manufacturer, model)) {
                    if (!platform.onSwitchFailed(vid, pid, attempt++)) {
                        error("AOA switch failed for %04X:%04X (driver/permission problem, see README
§3.7 / §3.8)".format(vid, pid))
                    }
                }
            } finally {
                LibUsb.unrefDevice(phone)
            }
```

```
            // 2. Phone disconnects and re-enumerates as 18D1:2D0x. Wait up to 10 s
            val deadline = System.currentTimeMillis() + 10_000
            while (findDevice(isAoa)?.also { LibUsb.unrefDevice(it) } == null) {
                if (System.currentTimeMillis() > deadline) {
                    error("Phone did not re-appear in AOA mode. Unlock it, accept the accessory prompt,
disable USB tethering.")
                }
                Thread.sleep(200)
            }
        }
        Thread.sleep(platform.settleDelayMs)

        // 3. Open + claim with OS-specific recovery
        val t = openAccessory()
        transport = t

        // 4. Encrypted channel (host = initiator)
        val channel = SecureQtSdk.createChannel(t)
        if (!channel.performHandshake(isInitiator = true)) {
            close()
            error("Handshake failed: the Android app didn't respond within ~30 s")
        }
        return SecureQtSdk.createTransferClient(channel).also { client = it }
    }

    private fun openAccessory(): DesktopAoaTransport {
        var lastError = LibUsb.ERROR_NO_DEVICE
        for (attempt in 1..MAX_OPEN_ATTEMPTS) {
            val dev = findDevice(isAoa)
            if (dev == null) { Thread.sleep(500); continue }
            val (vid, pid) = ids(dev)
            try {
                val handle = DeviceHandle()
                lastError = LibUsb.open(dev, handle)
                if (lastError == LibUsb.SUCCESS) {
                    val ep = SecureQtSdk.findDesktopBulkEndpoints(dev)
                    if (ep == null) {
                        lastError = LibUsb.ERROR_NOT_FOUND
                    } else {
                        // Linux: unbind any kernel driver. Windows: not supported, skip.
                        if (LibUsb.hasCapability(LibUsb.CAP_SUPPORTS_DETACH_KERNEL_DRIVER)) {
                            LibUsb.setAutoDetachKernelDriver(handle, true)
                        }
                        lastError = LibUsb.claimInterface(handle, ep.interfaceNumber)
                        if (lastError == LibUsb.SUCCESS) {
                            return SecureQtSdk.createDesktopTransport(handle, ep.inAddr, ep.outAddr, ep.i
nterfaceNumber)
                        }
                    }
                    LibUsb.close(handle)
                }
            } finally {
                LibUsb.unrefDevice(dev)
            }
            if (!platform.onOpenFailed(vid, pid, lastError, attempt)) break
            Thread.sleep(500)
        }
        error("Cannot open AOA device: ${LibUsb.strError(lastError)} ($lastError)")
    }

    /** Graceful disconnect: tell the phone, then release USB. */
    suspend fun disconnect() {
        runCatching { client?.sendDisconnect() }
        close()
```

```
    }

    override fun close() {
        transport?.close()        // releases interface, re-attaches kernel driver (Linux), closes handle
        transport = null
        client = null
    }

    /** Call once when the app exits. */
    fun shutdown() {
        close()
        LibUsb.exit(ctx)
    }

    // ── helpers ──
    private val isAoa: (Int, Int) -> Boolean = { vid, pid -> vid == AOA_VID && pid in AOA_PIDS }

    private fun ids(device: Device): Pair<Int, Int> {
        val d = DeviceDescriptor()
        LibUsb.getDeviceDescriptor(device, d)
        return (d.idVendor().toInt() and 0xFFFF) to (d.idProduct().toInt() and 0xFFFF)
    }

    /** Returns a referenced device (caller must unrefDevice) or null. */
    private fun findDevice(match: (vid: Int, pid: Int) -> Boolean): Device? {
        val list = DeviceList()
        if (LibUsb.getDeviceList(ctx, list) < 0) return null
        try {
            for (dev in list) {
                val (vid, pid) = ids(dev)
                if (match(vid, pid)) return LibUsb.refDevice(dev)
            }
        } finally {
            LibUsb.freeDeviceList(list, true)
        }
        return null
    }
}
```

Using it (same code on Windows and Linux)

```
import kotlinx.coroutines.runBlocking
import java.io.File

fun main() = runBlocking {
    val host = SecureQtHost()                   // picks WindowsHostPlatform / LinuxHostPlatform
automatically
    try {
        val client = host.connect()
        client.sendDeviceId("Workstation-01")
        client.sendFile(File("firmware.bin"), "/sdcard/Download") { sent, total ->
            print("\rUploading ${sent * 100 / maxOf(total, 1)}%")
        }
        println()
        client.listRemoteFiles("/sdcard/Download").forEach { println(it.name) }
        host.disconnect()
    } finally {
        host.shutdown()
    }
}
```

> **Threading:** `connect()` calls `Thread.sleep` while it waits for the device to re-enumerate. In a UI app, call it from `withContext(Dispatchers.IO) { host.connect() }`.

3.6 Host dependencies per OS

```
dependencies {
    implementation(files("libs/secureqt-sdk-jvm.jar"))   // includes usb4java and its natives

    // Windows only (needed by WindowsHostPlatform for automatic WinUSB install).
    // Safe to ship on Linux too: the installer reports NotWindows there.
    implementation(files("libs/usb-driver-installer-standalone.jar"))
}
```

If you don't ship the driver installer, pass `WindowsHostPlatform(installer = null)`. Users then install WinUSB once with Zadig (§3.7).

> The driver installer has its own guide: `usb-driver-installer/README.md` (PDF: `usb-driver-installer/USB_Driver_Installer_Guide.pdf`). It covers the full API, the managed state machine for UI dialogs, what an install changes on the PC, and security considerations for corporate machines.

3.7 Windows setup

One-time per PC (per phone model):

1. **Drivers.** `WindowsHostPlatform` installs WinUSB automatically through `usb-driver-installer` and shows one UAC prompt. To do it by hand with **Zadig**:
   - Make sure step 2 is done first, then run the app once so the phone switches to AOA. The app's open step fails at this point, which is expected.
   - In Zadig choose **Options → List All Devices**, pick `18D1:2D00` (or `18D1:2D01` **Interface 0**), select **WinUSB** and click **Install Driver**. The device usually appears as "Android Accessory Interface".
   - Run the app again.
2. **Phone in normal mode.** `switchDeviceToAoa` needs to open the phone before it switches. That works if **USB debugging is on** and the Google/OEM ADB driver is installed (that interface is WinUSB-based). Otherwise install WinUSB for the phone's own VID:PID; this hides the phone from Explorer/MTP while installed.
3. **adb.** Close Android Studio's device monitor, or run `adb kill-server`. `WindowsHostPlatform.prepare()` does this automatically.

Verify:

```
# Device Manager → "Universal Serial Bus devices" should list the accessory with driver "WinUSB"
pnputil /enum-devices /connected | findstr /i "VID_18D1"
```

Windows-specific troubleshooting:

Symptom	Fix
`open` → `-12` `ERROR_NOT_SUPPORTED` on `18D1:2D00/2D01`	WinUSB not installed, or Plug and Play binding hasn't finished. Retries handle the binding delay; otherwise install the driver

Symptom	Fix
`open` → `-3 ERROR_ACCESS`	`adb.exe` or another tool holds the device. Kill `adb`, close OEM suites (Samsung Smart Switch, etc.)
Phone reappears as `2D01` and open fails	USB debugging on gives the composite PID. WinUSB must be on **MI_00** (interface 0). Or turn USB debugging off to get `2D00`
AOA switch returns `false`	No WinUSB-compatible interface on the phone in normal mode. Turn USB debugging on, or allow the driver install on the phone VID:PID
Installer returns `UnsignedDriverBlocked`	Corporate policy blocks the driver. Ask IT to pre-install the WinUSB package
Works once, fails after replug	A different USB port can mean a new device instance on some PCs. Run the install again for that instance

3.8 Linux setup

One-time per PC. Create `/etc/udev/rules.d/51-secureqt.rules`:

```
# Google AOA accessory (18D1:2D00-2D05) and Google devices
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", MODE="0660", GROUP="plugdev", TAG+="uaccess"
# Your phones in normal mode: add one line per vendor (examples: Samsung, Xiaomi, OnePlus, Motorola)
SUBSYSTEM=="usb", ATTR{idVendor}=="04e8", MODE="0660", GROUP="plugdev", TAG+="uaccess"
SUBSYSTEM=="usb", ATTR{idVendor}=="2717", MODE="0660", GROUP="plugdev", TAG+="uaccess"
SUBSYSTEM=="usb", ATTR{idVendor}=="2a70", MODE="0660", GROUP="plugdev", TAG+="uaccess"
SUBSYSTEM=="usb", ATTR{idVendor}=="22b8", MODE="0660", GROUP="plugdev", TAG+="uaccess"
```

Then:

```
sudo groupadd -f plugdev
sudo usermod -aG plugdev "$USER"          # log out and back in for the group to apply
sudo udevadm control --reload-rules && sudo udevadm trigger
# unplug and replug the phone
```

`TAG+="uaccess"` gives the logged-in desktop user access without the group. `GROUP="plugdev"` covers SSH and headless sessions.

Verify:

```
lsusb | grep -i 18d1                      # after connect: "18d1:2d00 Google Inc." (or 2d01)
ls -l /dev/bus/usb/$(lsusb -d 18d1: | awk '{print $2"/"substr($4,1,3)}')
                                          # should show group plugdev and rw for the group, or a '+' ACL
```

Linux-specific troubleshooting:

Symptom	Fix
`open` → `-3 ERROR_ACCESS`	udev rule missing, or the vendor ID isn't listed. Add it, reload, replug. Running once with `sudo` confirms it's a permission problem
`claimInterface` → `-6 ERROR_BUSY`	A kernel driver or another process has the interface. `setAutoDetachKernelDriver` handles the kernel case; check `lsof /dev/bus/usb/BBB/DDD` for processes
AOA switch works, then the device keeps disconnecting	ModemManager probing the phone. Add `ENV{ID_MM_DEVICE_IGNORE}="1"` to the rule lines, or `sudo systemctl stop ModemManager`

Symptom	Fix
Phone vanishes from the file manager while connected	Expected: in AOA mode the phone stops exposing MTP until it leaves accessory mode (unplug)
`UnsatisfiedLinkError: libusb4java`	usb4java 1.3.0 has no native for your OS and CPU (for example macOS on Apple Silicon or Windows on ARM). Use an x86-64 JDK, or build `libusb4java` for your platform

3.9 Shipping the host app

	Windows	Linux
Package	`jpackage --type msi` / `exe` (or Compose Desktop `packageMsi`)	`jpackage --type deb` / `rpm` (or `packageDeb`)
Install-time step	Optional: pre-install WinUSB for `18D1:2D00` and `18D1:2D01 MI_00` from the installer so users never see a UAC prompt	Install `51-secureqt.rules` to `/etc/udev/rules.d/` in the package's post-install script and run `udevadm control --reload-rules && udevadm trigger`
Runtime privileges	Normal user	Normal user (member of `plugdev` or logged-in seat)

3.10 Transfer client: every function

`SecureQtSdk.createTransferClient(channel)` returns an `ISecureTransferClient`. It needs a channel that has finished `performHandshake`. With the session manager, use `session.client` instead: it's already connected.

Calls run one at a time, so you can call the client from several coroutines. Paths below use `/sdcard/...`, which works with both the sample Android app and the §4.5 command loop. See Remote paths.

```
val client = SecureQtSdk.createTransferClient(channel)
val onProgress = { done: Long, total: Long -> println("$done of $total bytes") }

// Browse and manage remote files
val files = client.listRemoteFiles("/sdcard/Download")
val created = client.createFolder("/sdcard/Download/maps")
val renamed = client.renameFile("/sdcard/Download/old.map", "new.map")
val deleted = client.deleteFile("/sdcard/Download/new.map")

// Send to the phone
client.sendFile(File("city.map"), "/sdcard/Download/maps", onProgress = onProgress)
client.sendFile(File("city.map"), "/sdcard/Download/maps", "city-v2.map")   // save under another name
client.sendDirectory(File("mapsFolder"), "/sdcard/Download", onProgress)    // streamed as a ZIP

// Fetch from the phone
client.fetchFile("/sdcard/Download/maps/city.map", File("downloads"), onProgress)
client.fetchRemoteDirectory("/sdcard/Download/maps", File("backup"), onProgress)

// Identify this host
client.sendDeviceId("desktop-01")

// Cancel a running transfer, for example from a Cancel button
val transfer = scope.launch { client.sendFile(File("big.map"), "/sdcard/Download") }
client.cancelTransfer()
transfer.join()
```

```
// Your own command (see §6), with a timeout in case the client has no handler
val reply = withTimeoutOrNull(5_000) { client.executeCustomCommand(0x10) }

// Finish the session, then close the transport
client.sendDisconnect()
```

Function	Standard command	Client replies	Returns	Watch out for
`listRemoteFiles(path)`	`CMD_LIST_DIR`	Count, then one packet per entry	`List<SdkRemoteFile>`	An empty list can also mean a missing folder or a failed call. `lastModified` is always `0`
`sendFile(localFile, remotePath, remoteFileName?, onProgress?)`	`CMD_SEND_FILE_START`	Nothing	`true` when all bytes were written to USB	`true` **doesn't mean saved.** Check with `listRemoteFiles`. `remotePath` is the folder. An existing file is overwritten by the sample app
`sendDirectory(localDir, remotePath, onProgress?)`	`CMD_SEND_DIR_ZIP_START`	Nothing	As `sendFile`	Returns before the client finishes unzipping. `total` in `onProgress` is `0`
`fetchFile(remotePath, localSaveDir, onProgress?)`	`CMD_FETCH_FILE_START`	Size, data, `TYPE_EOF`	`true` when the stream ended	**A missing remote file returns `true` and leaves a 0-byte local file.** A dropped connection also returns `true` with a partial file. Compare against the size from `listRemoteFiles`
`fetchRemoteDirectory(remotePath, localSaveDir, onProgress?)`	`CMD_FETCH_DIR_ZIP_START`	Size, ZIP data, `TYPE_EOF`	`true` when the stream ended	Saves to `localSaveDir/<folder name>`. Only use with a client you trust, because ZIP entry names aren't checked for `../`
`createFolder(remotePath)`	`CMD_CREATE_FOLDER`	`[0x01]` or `[0x00]`	`true` if created	The sample app returns `false` if the folder already exists
`renameFile(remotePath, newName)`	`CMD_RENAME_FILE`	`[0x01]` or `[0x00]`	`true` if renamed	`newName` is a name, not a path
`deleteFile(remotePath)`	`CMD_DELETE_FILE`	`[0x01]` or `[0x00]`	`true` if deleted	Also works on folders with the sample app
`sendDeviceId(deviceId)`	`CMD_SET_DEVICE_ID`	Nothing	`true` when written to USB	The session manager and `DesktopAoaLink` already send this on connect
`sendDisconnect()`	`CMD_DISCONNECT`	Nothing	Nothing	Wipes the session keys but leaves USB open, so still call `transport.close()`. Later calls return `false` or an empty list
`cancelTransfer()`	Raw `TYPE_CANCEL`	Nothing	Nothing	The only function that isn't `suspend`. The running transfer returns `false`
`executeCustomCommand(commandId, payload)`	Your own ID (§6)	Exactly one packet	Reply, or `null`	**Waits forever if the client has no handler.** Wrap it in `withTimeoutOrNull`

`onProgress` receives `(bytesDone, total)` about every 250 ms. It isn't called for a 0-byte file.

Timeouts. Only calls that wait for a short reply can time out: `listRemoteFiles`, `createFolder`, `renameFile`, `deleteFile` and `executeCustomCommand`. Transfers ignore coroutine cancellation, so stop them with `cancelTransfer()`. If a reply arrives after its timeout, the next call reads it as its own reply, so reconnect after a timeout.

Failures. A cancel packet from the client makes the calls that wait for a reply throw `SecureChannel.TransferCancelledException`.

Remote paths

The client decides what a path means, not the SDK:

Path sent	Sample Android app	§4.5 command loop
`/sdcard/Download`	`<external storage>/Download`	`/sdcard/Download`
`/Download`	`<external storage>/Download`	Root-level `/Download`, which fails
`/storage/emulated/0/Download`	`<external storage>/storage/emulated/0/Download`, the wrong folder	`/storage/emulated/0/Download`

Use `/sdcard/...` unless your client resolves paths differently. In the sample app, paths under `/sdcard/Android/data/com.example.smartnav_v3` and `/sdcard/SmartNavV3` go through the SmartNav storage provider.

3.11 Sending standard commands with `sendSecure`

The client functions above already send the standard commands, so most projects don't need this. Use `channel.sendSecure` directly only to control the order of packets yourself.

```
import com.example.secureqt.sdk.protocol.CommandPayloadBuilder
import com.example.secureqt.sdk.protocol.StandardCommandIds

val command = CommandPayloadBuilder.buildWithString(StandardCommandIds.CMD_CREATE_FOLDER, "/sdcard/
Download/maps")
if (channel.sendSecure(command)) {
    val created = channel.receiveSecure()?.firstOrNull() == 1.toByte()
}
```

Every standard command uses one builder, and its reply decides what to read next:

Command	Builder	Read after sending
`CMD_LIST_DIR`	`buildWithString(cmd, path)`	One `receiveSecure()` for the count, then one per entry
`CMD_SEND_FILE_START`	`buildFileStart(cmd, fullPath, size)`	Nothing. Send data, then `TYPE_EOF`
`CMD_SEND_DIR_ZIP_START`	`buildFileStart(cmd, fullPath, 0L)`	Nothing. Send ZIP data, then `TYPE_EOF`
`CMD_FETCH_FILE_START`	`buildWithString(cmd, path)`	One `receiveSecure()` for the size, then `receiveRawPacket()` until `TYPE_EOF`
`CMD_FETCH_DIR_ZIP_START`	`buildWithString(cmd, path)`	Same as `CMD_FETCH_FILE_START`, with ZIP data
`CMD_DELETE_FILE`, `CMD_CREATE_FOLDER`	`buildWithString(cmd, path)`	One `receiveSecure()`: `[0x01]` or `[0x00]`

Command	Builder	Read after sending
`CMD_RENAME_FILE`	`buildWithTwoStrings(cmd, path, newName)`	One `receiveSecure()`: `[0x01]` or `[0x00]`
`CMD_SET_DEVICE_ID`	`buildWithString(cmd, id)`	Nothing. Calling `receiveSecure()` here would hang
`CMD_DISCONNECT`	`buildSimple(cmd)`	Nothing

Three rules keep the channel in sync:

- **Read exactly what each command returns.** An unread reply gets mistaken for the reply to your next command.
- **Don't use `sendSecure` while a client call is running.** The client's lock doesn't cover direct channel calls, so their packets and replies would mix. Wait for the client call to return first.
- **Don't use `executeCustomCommand` for a standard command with no reply.** It waits for a reply that never comes.

`DesktopUsbSession` doesn't expose its channel, so with the session manager use `session.client`. To reach the channel, use the manual flow in §3.3 or create a `DesktopAoaLink`, which has a public `channel`.

3.12 Example: sending a `.map` file end to end

The SDK treats a `.map` file like any other binary file, so there is no special command or size limit. This example sends `city.map` (1,048,699 bytes) to `/sdcard/Download/maps` on the phone.

Before you start:

- The phone is connected and the handshake has succeeded (§3.3 steps 1–4, or `session.connect()`).
- The Android app is running its command loop (§4.5). `CMD_SEND_FILE_START` needs no `.map`-specific code.
- The Android app has storage permission. Without it, the sample app discards the data and the host still gets `true` from `sendFile`.

Option A: with the client (recommended)

```
import com.example.secureqt.sdk.SecureQtSdk
import com.example.secureqt.sdk.channel.SecureChannel
import com.example.secureqt.sdk.client.ISecureTransferClient
import com.example.secureqt.sdk.protocol.CommandPayloadBuilder
import com.example.secureqt.sdk.protocol.StandardCommandIds
import java.io.File
import java.io.FileInputStream
import java.nio.ByteBuffer

suspend fun sendMapFile(
    client: ISecureTransferClient,
    mapFile: File,
    remoteDir: String,
    onProgress: (Int) -> Unit = {}
): Boolean {
    require(mapFile.isFile && mapFile.extension.equals("map", ignoreCase = true)) { "Not a .map file: $ma
pFile" }

    // false can simply mean "already exists", so the check below decides the outcome
    client.createFolder(remoteDir)

    val sent = client.sendFile(mapFile, remoteDir) { done, total ->
        if (total > 0) onProgress((done * 100 / total).toInt())
    }
    if (!sent) return false
```

```
    // sendFile has no acknowledgement. The client handles commands in order, so this
    // reply only arrives after it has finished writing the file.
    val saved = client.listRemoteFiles(remoteDir).firstOrNull { it.name == mapFile.name }
    return saved != null && saved.size == mapFile.length()
}
```

Call it from a coroutine:

```
val client = SecureQtSdk.createTransferClient(channel)     // or session.client

val saved = sendMapFile(client, File("maps/city.map"), "/sdcard/Download/maps") { percent ->
    println("Uploading: $percent%")
}
println(if (saved) "Map saved on the phone" else "Map transfer failed")
```

The size check confirms that the right number of bytes was saved, not that they match. For a content check, add a custom command (§6) that returns the file's SHA-256 from the phone.

Option B: with `sendSecure`

This sends the same bytes as Option A, plus the device ID, using the channel directly:

```
suspend fun sendMapFileWithSendSecure(
    channel: SecureChannel,
    mapFile: File,
    remoteDir: String,
    hostDeviceId: String
): Boolean {
    require(mapFile.isFile) { "File not found: $mapFile" }
    val remotePath = "$remoteDir/${mapFile.name}"

    // 1. CMD_SET_DEVICE_ID: no reply, so do not call receiveSecure()
    val setId = CommandPayloadBuilder.buildWithString(StandardCommandIds.CMD_SET_DEVICE_ID, hostDeviceId)
    if (!channel.sendSecure(setId)) return false

    // 2. CMD_CREATE_FOLDER: one reply, [0x01] or [0x00]
    val mkdir = CommandPayloadBuilder.buildWithString(StandardCommandIds.CMD_CREATE_FOLDER, remoteDir)
    if (!channel.sendSecure(mkdir)) return false
    channel.receiveSecure() ?: return false

    // 3. CMD_SEND_FILE_START: no reply, the file data follows straight away
    val start = CommandPayloadBuilder.buildFileStart(
        StandardCommandIds.CMD_SEND_FILE_START, remotePath, mapFile.length()
    )
    if (!channel.sendSecure(start)) return false

    // 4. File data: one encrypted TYPE_DATA packet per 256 KB chunk
    var ok = true
    try {
        FileInputStream(mapFile).use { input ->
            val buffer = ByteArray(256 * 1024)
            while (ok) {
                val read = input.read(buffer)
                if (read == -1) break
                ok = channel.sendSecure(buffer.copyOf(read))
            }
        }
    } catch (e: java.io.IOException) {
        ok = false
    }
```

```
    // 5. End of stream: raw, unencrypted control packet
    val endType = if (ok) SecureQtSdk.Packets.TYPE_EOF else SecureQtSdk.Packets.TYPE_CANCEL
    if (!channel.sendRawPacket(endType, ByteArray(0)) || !ok) return false

    // 6. CMD_LIST_DIR: [4: count], then one packet per entry. Read them all.
    val list = CommandPayloadBuilder.buildWithString(StandardCommandIds.CMD_LIST_DIR, remoteDir)
    if (!channel.sendSecure(list)) return false
    val count = ByteBuffer.wrap(channel.receiveSecure() ?: return false).int
    var savedSize = -1L
    repeat(count) {
        val item = ByteBuffer.wrap(channel.receiveSecure() ?: return false)
        item.get()                                          // isDirectory
        val size = item.long
        val name = ByteArray(item.int).also { item.get(it) }.toString(Charsets.UTF_8)
        if (name == mapFile.name) savedSize = size
    }
    return savedSize == mapFile.length()
}
```

What goes over USB

These are the packets Option B sends for `city.map`. Option A sends the same packets without the first one.

#	Flow	Packet	Size on USB	Decrypted content
1	Host → phone	`TYPE_PUBLIC_KEY` (both directions, not encrypted)	96 bytes	ECDH public key. Done by `performHandshake`
2	Host → phone	`TYPE_DATA`	48 bytes	`CMD_SET_DEVICE_ID`, `"desktop-01"`
3	Host → phone	`TYPE_DATA`	59 bytes	`CMD_CREATE_FOLDER`, `"/sdcard/Download/maps"`
4	Phone → host	`TYPE_DATA`	34 bytes	`[0x01]`, or `[0x00]` if the folder exists
5	Host → phone	`TYPE_DATA`	76 bytes	`CMD_SEND_FILE_START`, full path, size `1048699`
6–9	Host → phone	4 × `TYPE_DATA`	262,177 bytes each	256 KB of file data per packet
10	Host → phone	`TYPE_DATA`	156 bytes	Last 123 bytes of file data
11	Host → phone	`TYPE_EOF` (not encrypted)	5 bytes	None. The phone closes the file
12	Host → phone	`TYPE_DATA`	59 bytes	`CMD_LIST_DIR`, `"/sdcard/Download/maps"`
13	Phone → host	`TYPE_DATA`	37 bytes	Entry count
14	Phone → host	`TYPE_DATA`	One per entry	`isDir`, size, name. Must show `city.map`, `1048699`

An encrypted packet is 33 bytes larger than its content: a 5-byte header, a 12-byte IV and a 16-byte GCM tag. Every packet has a new IV.

Decrypted content of packets 2, 3 and 5, with integers in big-endian:

```
2  09 | 00 00 00 0A | "desktop-01"
3  08 | 00 00 00 15 | "/sdcard/Download/maps"
5  01 | 00 00 00 1E | "/sdcard/Download/maps/city.map" | 00 00 00 00 00 10 00 7B
   cmd  name length   name                                file size (1048699)
```

If something goes wrong

What you see	Likely cause	What to do
`sendFile` returns `true`, but the size check fails	The phone couldn't write the file: no storage permission, a bad path, or a full disk	Check the permission and path, then send again
The file is in an unexpected folder on the phone	A `/storage/emulated/0/...` path sent to the sample app	Use `/sdcard/...`. See Remote paths
The call never returns	The client isn't running its command loop, or there's an unread or missing reply	Check the Android logs and the rules in §3.11
`sendFile` returns `false`	Missing local file, cancelled transfer, or lost USB connection	Check the file, reconnect if needed, and send the whole file again. Transfers can't resume

---

4. Android (AAR) integration

4.1 Add dependencies

```
// app/build.gradle.kts
dependencies {
    implementation(files("libs/secureqt-sdk-android.aar"))
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.10.2")
}
```

Requirements: `minSdk 24`, Java 11 compile options.

4.2 Accessory filter

The `manufacturer` / `model` here **must match** what the desktop passes to `switchDeviceToAoa()`.

`res/xml/accessory_filter.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <usb-accessory
        manufacturer="AndroidOpenAccessory"
        model="DataTransfer"
        version="1.0" />
</resources>
```

4.3 Manifest

```
<manifest ...>
    <uses-feature android:name="android.hardware.usb.accessory" android:required="true" />
```

```
    <application ...>
        <activity android:name=".MainActivity" android:exported="true" android:launchMode="singleTask">
            <intent-filter>
                <action android:name="android.hardware.usb.action.USB_ACCESSORY_ATTACHED" />
            </intent-filter>
            <meta-data
                android:name="android.hardware.usb.action.USB_ACCESSORY_ATTACHED"
                android:resource="@xml/accessory_filter" />
        </activity>
    </application>
</manifest>
```

When the activity is launched by `USB_ACCESSORY_ATTACHED`, the user's permission is granted implicitly. If you open the accessory any other way, call `UsbManager.requestPermission(accessory, pendingIntent)` first.

4.4 Open the transport & handshake

```
import android.hardware.usb.UsbAccessory
import android.hardware.usb.UsbManager
import com.example.secureqt.sdk.SecureQtSdk
import com.example.secureqt.sdk.createAndroidAccessoryTransport

val usbManager = getSystemService(USB_SERVICE) as UsbManager
val accessory: UsbAccessory = intent.getParcelableExtra(UsbManager.EXTRA_ACCESSORY)
    ?: usbManager.accessoryList?.firstOrNull()
    ?: return

lifecycleScope.launch(Dispatchers.IO) {
    val transport = SecureQtSdk.createAndroidAccessoryTransport(usbManager, accessory)
        ?: return@launch   // null → no permission or accessory already open

    val channel = SecureQtSdk.createChannel(transport)
    if (!channel.performHandshake(isInitiator = false)) {   // Android is the responder
        transport.close(); return@launch
    }

    runCommandLoop(channel)   // §4.5
    transport.close()
}
```

> Run the session in a **foreground service** for long transfers so Android doesn't kill it when the screen turns off.

4.5 Implement the command loop

The Android side must answer each command in the exact format the desktop client expects (see §5). Minimal example:

```
import com.example.secureqt.sdk.SecureQtSdk
import com.example.secureqt.sdk.channel.SecureChannel
import com.example.secureqt.sdk.protocol.CommandPayloadParser
import com.example.secureqt.sdk.protocol.StandardCommandIds
import java.io.File
import java.io.FileInputStream
import java.io.FileOutputStream
import java.nio.ByteBuffer
```

```
suspend fun runCommandLoop(channel: SecureChannel) {
    while (true) {
        val raw = try {
            channel.receiveSecure() ?: break           // null → disconnected
        } catch (e: SecureChannel.TransferCancelledException) {
            continue                                   // host cancelled; keep listening
        }
        val cmd = CommandPayloadParser.parse(raw) ?: continue

        when (cmd.commandId) {
            StandardCommandIds.CMD_LIST_DIR -> {
                val dir = File(CommandPayloadParser.readString(cmd.payload))
                val items = dir.listFiles().orEmpty()
                channel.sendSecure(ByteBuffer.allocate(4).putInt(items.size).array())
                for (f in items) {
                    val name = f.name.toByteArray(Charsets.UTF_8)
                    channel.sendSecure(
                        ByteBuffer.allocate(1 + 8 + 4 + name.size)
                            .put(if (f.isDirectory) 1.toByte() else 0.toByte())
                            .putLong(if (f.isFile) f.length() else 0L)
                            .putInt(name.size).put(name)
                            .array()
                    )
                }
            }

            StandardCommandIds.CMD_SEND_FILE_START -> {
                val (path, size) = CommandPayloadParser.readFileStart(cmd.payload)
                val out = File(path).apply { parentFile?.mkdirs() }
                FileOutputStream(out).use { fos ->
                    var received = 0L
                    while (true) {
                        val (type, body) = channel.receiveRawPacket() ?: break
                        if (type == SecureQtSdk.Packets.TYPE_CANCEL) { out.delete(); break }
                        if (type != SecureQtSdk.Packets.TYPE_DATA) break        // TYPE_EOF
                        val chunk = channel.decryptData(body) ?: break
                        fos.write(chunk); received += chunk.size
                    }
                }
            }

            StandardCommandIds.CMD_FETCH_FILE_START -> {
                val f = File(CommandPayloadParser.readString(cmd.payload))
                val size = if (f.isFile) f.length() else 0L
                channel.sendSecure(ByteBuffer.allocate(8).putLong(size).array())
                if (f.isFile) FileInputStream(f).use { input ->
                    val buf = ByteArray(256 * 1024)
                    while (true) {
                        val n = input.read(buf); if (n == -1) break
                        channel.sendSecure(buf.copyOf(n))
                    }
                }
                channel.sendRawPacket(SecureQtSdk.Packets.TYPE_EOF, ByteArray(0))
            }

            StandardCommandIds.CMD_DELETE_FILE -> {
                val ok = File(CommandPayloadParser.readString(cmd.payload)).deleteRecursively()
                channel.sendSecure(byteArrayOf(if (ok) 1.toByte() else 0.toByte()))
            }

            StandardCommandIds.CMD_RENAME_FILE -> {
                val (oldPath, newName) = CommandPayloadParser.readTwoStrings(cmd.payload)
                val src = File(oldPath)
                val ok = src.renameTo(File(src.parentFile, newName))
```

```
                channel.sendSecure(byteArrayOf(if (ok) 1.toByte() else 0.toByte()))
            }

            StandardCommandIds.CMD_CREATE_FOLDER -> {
                val ok = File(CommandPayloadParser.readString(cmd.payload)).mkdirs()
                channel.sendSecure(byteArrayOf(if (ok) 1.toByte() else 0.toByte()))
            }

            StandardCommandIds.CMD_SET_DEVICE_ID -> {
                val id = CommandPayloadParser.readString(cmd.payload)   // no reply expected
            }

            StandardCommandIds.CMD_DISCONNECT -> break

            // CMD_SEND_DIR_ZIP_START / CMD_FETCH_DIR_ZIP_START: same as file send/fetch,
            // but the stream is a ZIP (unzip on receive / zip on send).
        }
    }
    channel.disconnect()
}
```

For large files, use a read → decrypt → write pipeline with coroutine `Channel`s; `SecureTransferClientImpl` does this on the host side. `encryptAndWrapData()` + `sendPrebuiltPacket()` let you encrypt on `Dispatchers.Default` and send on `Dispatchers.IO` in parallel.

---

5. Standard commands & wire contract

Every command is sent as **one encrypted** `TYPE_DATA` **packet**: `[1: commandId][args]`. All integers are **big-endian**; strings are `[4: byteLen][UTF-8 bytes]`.

ID	Constant	Host sends (after cmd byte)	Android must reply
`0x00`	`CMD_LIST_DIR`	`str path`	secure `[4: count]`, then `count` × secure `[1: isDir][8: size][4: nameLen][name]`
`0x01`	`CMD_SEND_FILE_START`	`str fullPath` `[8: size]`, then `TYPE_DATA` … then `TYPE_EOF` (or `TYPE_CANCEL`)	nothing
`0x02`	`CMD_FETCH_FILE_START`	`str path`	secure `[8: size]` (0 if missing), then `TYPE_DATA` …, then raw `TYPE_EOF`
`0x03`	`CMD_FETCH_DIR_ZIP_START`	`str path`	secure `[8: size]`, then ZIP stream as `TYPE_DATA` …, then `TYPE_EOF`
`0x04`	`CMD_DISCONNECT`	–	nothing (close session)
`0x05`	`CMD_SEND_DIR_ZIP_START`	`str fullPath` `[8: 0]`, then ZIP stream, then `TYPE_EOF`	nothing
`0x06`	`CMD_DELETE_FILE`	`str path`	secure `[1: 0x01 ok / 0x00 fail]`
`0x07`	`CMD_RENAME_FILE`	`str oldPath` `str newName`	secure `[1: ok]`
`0x08`	`CMD_CREATE_FOLDER`	`str path`	secure `[1: ok]`
`0x09`	`CMD_SET_DEVICE_ID`	`str deviceId`	nothing

Outer packet types (`SecureQtSdk.Packets`):

Byte	Name	Encrypted?	Meaning
`0x01`	`TYPE_PUBLIC_KEY`	No	ECDH public key (handshake)
`0x02`	`TYPE_DATA`	**Yes** (`[12: IV][ciphertext+16: GCM tag]`)	Command / response / file chunk
`0x03`	`TYPE_EOF`	No	End of stream
`0x04`	`TYPE_PING`	No	Keep-alive (ignored by `receiveSecure`)
`0x05`	`TYPE_ERROR`	No	Error
`0x06`	`TYPE_CANCEL`	No	Transfer aborted

Frame layout: `[1: type][4: payloadLength][payload]` (`SecureQtSdk.Codec.HEADER_SIZE == 5`).

---

6. Custom commands

Use IDs that don't collide with the standard set (for example `0x10` and up). Add new commands without changing the SDK:

Host (JAR):

```
const val CMD_GET_APP_VERSION: Byte = 0x10
const val CMD_ECHO: Byte = 0x11

// No arguments
val reply: ByteArray? = client.executeCustomCommand(CMD_GET_APP_VERSION)
val version = reply?.toString(Charsets.UTF_8)

// With arguments: pass ONLY the arguments; executeCustomCommand prepends commandId itself
val text = "hello".toByteArray()
val args = java.nio.ByteBuffer.allocate(4 + text.size).putInt(text.size).put(text).array()
val echo = client.executeCustomCommand(CMD_ECHO, args)
```

> ⚠️ Don't pass the output of `CommandPayloadBuilder.build*()` to `executeCustomCommand`. Those builders already include the command byte, so the ID would be sent twice. Use the builders with `channel.sendSecure(...)` directly instead.

Android (AAR): reply with **exactly one** `sendSecure()`, because `executeCustomCommand` reads a single response:

```
when (cmd.commandId) {
    0x10.toByte() -> channel.sendSecure(BuildConfig.VERSION_NAME.toByteArray())
    0x11.toByte() -> channel.sendSecure(CommandPayloadParser.readString(cmd.payload).toByteArray())
}
```

Encoding helpers (`com.example.secureqt.sdk.protocol.CommandPayloadBuilder`, not exposed on the `SecureQtSdk` facade, so import it directly):

Builder	Wire format	Matching parser (`CommandPayloadParser`)
`buildSimple(cmd)`	`[cmd]`	`parse()`
`buildWithString(cmd, s)`	`[cmd][4][s]`	`readString()`
`buildFileStart(cmd, path, size)`	`[cmd][4][path][8]`	`readFileStart()`

Builder	Wire format	Matching parser (`CommandPayloadParser`)
`buildWithTwoStrings(cmd, a, b)`	`[cmd][4][a][4][b]`	`readTwoStrings()`
`buildCustom(cmd, vararg segs)`	`[cmd]([4][seg])*`	`readString()` / manual

Other readers: `readInt()`, `readLong()`, `readBoolean()`.

Example: getting the client app version

This example has the host ask the Android app for its version. The SDK needs no changes because it only carries the bytes. The host and the app must use the same command ID.

Step 1: add the host function (JAR). The timeout matters: if the app has no handler for this command, it sends no reply and the call would otherwise wait forever.

```
import com.example.secureqt.sdk.client.ISecureTransferClient
import kotlinx.coroutines.withTimeoutOrNull

const val CMD_GET_APP_VERSION: Byte = 0x10

suspend fun getClientAppVersion(client: ISecureTransferClient): String? {
    val reply = withTimeoutOrNull(5_000) {
        client.executeCustomCommand(CMD_GET_APP_VERSION)
    }
    return reply?.toString(Charsets.UTF_8)
}
```

Step 2: call it after connecting.

```
// With the session manager
val session = sessions.getSession(deviceId) ?: return
if (session.connect()) {
    val version = getClientAppVersion(session.client)
    println("Client app version: ${version ?: "no reply"}")
}

// With the manual flow (§3.3)
val version = getClientAppVersion(SecureQtSdk.createTransferClient(channel))
```

Step 3: reply from the Android app (AAR). Send exactly one `sendSecure()`. In a simple command loop (§4.5), add a branch:

```
when (cmd.commandId) {
    CMD_GET_APP_VERSION -> channel.sendSecure(BuildConfig.VERSION_NAME.toByteArray(Charsets.UTF_8))
    // ...standard commands
}
```

In the sample Android app, add a handler instead:

```
@Singleton
class GetAppVersionHandler @Inject constructor(
    @param:ApplicationContext private val context: Context,
    private val channel: SecureChannel
) : ICommandHandler {

    override val commandId: Byte = AppCommandIds.CMD_GET_APP_VERSION

    @Suppress("DEPRECATION")
    override suspend fun handle(payload: ByteBuffer, cmdCtx: CommandContext): Boolean {
```

```
        val version = context.packageManager
            .getPackageInfo(context.packageName, 0).versionName ?: "unknown"
        channel.sendSecure(version.toByteArray(Charsets.UTF_8))
        return true // keep the command loop running
    }
}
```

Then register it in `CommandHandlerModule`:

```
@Binds @IntoSet
abstract fun bindGetAppVersionHandler(impl: GetAppVersionHandler): ICommandHandler
```

What goes over USB:

Flow	Packet	Decrypted content
Host → phone	`TYPE_DATA`	`[0x10]`
Phone → host	`TYPE_DATA`	The version as UTF-8 text, for example `1.2.3`. No command ID

If something goes wrong:

What you see	Likely cause	What to do
`getClientAppVersion` returns `null` after 5 seconds	The app has no handler for `0x10`, or its command loop isn't running	Add and register the handler. The sample app defines `CMD_GET_APP_VERSION` but has no handler for it yet
A later call gets the wrong reply	A reply arrived after its timeout, or a handler sent more than one packet	Reconnect after a timeout, and send exactly one reply
The app reads the wrong command or arguments	The output of `CommandPayloadBuilder` was passed to `executeCustomCommand`, which sends the ID twice	Pass only the arguments

The sample app also has `CMD_GET_APP_DETAILS` (`0x11`), which returns the app name, package name, app version and OS version. Its reply is encoded by `ClientAppDetailsCodec`, which is part of the app and not the JAR, so another desktop project needs its own copy of the decoder.

---

7. API reference

`SecureQtSdk` (common)

Member	Description
`VERSION`	`"1.0.0"`
`createChannel(transport): SecureChannel`	Wrap any `IUsbTransport` in an encrypted channel
`createTransferClient(channel): ISecureTransferClient`	High-level host client
`createBuffer()`	Sliding window buffer (used internally)
`Packets` / `Codec`	Packet type constants / frame encoder–decoder
`CommandPayloadParser`	Command decoder
`Utils`	`FormatUtils`

JVM-only extensions (`import com.example.secureqt.sdk.*`)

Function	Description
`SecureQtSdk.switchDeviceToAoa(device, manufacturer, model): Boolean`	Sends AOA requests 51/52/53; the device then re-enumerates
`SecureQtSdk.findDesktopBulkEndpoints(device): BulkEndpoints?`	`interfaceNumber`, `inAddr`, `outAddr`
`SecureQtSdk.createDesktopTransport(handle, inEp, outEp, ifaceNum)`	`DesktopAoaTransport`; call `close()` when done
`SecureQtSdk.createDesktopSessionManager(deviceManager, logger)`	Session manager with the ready-made `DesktopUsbSession`; see §3.5

Desktop host layer (`com.example.secureqt.sdk.host`)

Type	Description
`DesktopUsbSessionManager<S, T>`	`startPolling()`, `stopPolling()`, `pollDevices()`, `close()`, `getSession(deviceId)`, `getActiveSessions()`, `sessionsState: StateFlow<Map<String, T>>`
`DesktopUsbSession`	`connect()`, `disconnect()`, `client`, `sessionState`. Implements `ManagedUsbSession`
`DesktopSessionState` / `DesktopSessionStatus`	`deviceId`, `deviceName`, `status`, `isAoaMode`. Status is `Disconnected`, `Connecting`, `Ready` or `Error(message)`
`ManagedUsbSession<T>`	Implement this to add your own per-device state to a session
`DesktopUsbDeviceManager`	Finds Android devices by physical port. `discoverDevices()` results hold a native reference that must be released; `cleanup()` shuts down libusb
`DesktopUsbDevice`	`id`, `busNumber`, `portNumber`, `vendorId`, `productId`, `isAoa`, `deviceName`, `device`
`DesktopUsbConnection`	Opens and claims the device, with driver-binding retries. Implements `IUsbTransport`. `killAdbOnWindows` defaults to `true`
`DesktopAoaLink`	The connect flow for one device: AOA switch, re-enumeration, open and handshake
`UsbDriverSupport`	Optional hook for installing missing drivers
`StdoutLogger`	Default `ILogger`; writes to standard output

Android-only extensions

Function	Description
`SecureQtSdk.createAndroidAccessoryTransport(usbManager, accessory)`	`AndroidAoaTransport?`; `null` if it can't be opened. Call `close()` when done

`SecureChannel`

Method	Description
`suspend performHandshake(isInitiator): Boolean`	ECDH exchange. Waits up to ~30 s; the initiator resends its key every 1 s
`suspend sendSecure(bytes): Boolean`	Encrypt + send one `TYPE_DATA` packet
`suspend receiveSecure(): ByteArray?`	Next decrypted `TYPE_DATA`; skips EOF/PING/unknown; throws `TransferCancelledException` on `TYPE_CANCEL`; `null` on disconnect

Method	Description
`suspend sendRawPacket(type, payload)` / `receiveRawPacket(timeoutMs)`	Unencrypted framed I/O (control packets, streaming)
`encryptAndWrapData(bytes)` + `sendPrebuiltPacket(packet)`	Split encrypt/send for parallel pipelines
`decryptData(payload)`	Decrypt a raw `TYPE_DATA` payload
`disconnect()`	Wipe session keys and buffers

`ISecureTransferClient` (host)

`listRemoteFiles`, `sendFile`, `sendDirectory`, `fetchFile`, `fetchRemoteDirectory`, `deleteFile`, `renameFile`, `createFolder`, `sendDeviceId`, `sendDisconnect`, `cancelTransfer`, `executeCustomCommand`. See §3.10 for what each one sends and returns. All except `cancelTransfer` are `suspend`. Progress callbacks are `(bytesDone: Long, total: Long) -> Unit`, throttled to about every 250 ms. For directory uploads `total` is `0` (unknown).

Custom transport

Implement `IUsbTransport` to run the protocol over any byte pipe (for example a loopback for tests):

```
interface IUsbTransport {
    suspend fun read(buffer: ByteArray): Int      // bytes read, 0 = nothing yet, -1 = disconnected
    suspend fun write(buffer: ByteArray): Boolean
    fun isConnected(): Boolean
    fun clearBuffer()
}
```

---

8. Limits, threading & gotchas

- **Max packet payload: 10 MB.** Larger frames are rejected. The client streams files in 256 KB chunks.
- **One command at a time.** `SecureTransferClientImpl` serialises calls with a mutex, so concurrent calls queue up.
- **Call from coroutines.** All I/O is `suspend` and switches to `Dispatchers.IO` internally. Don't call it from the Android main thread with `runBlocking`.
- **The host must claim the USB interface.** `createDesktopTransport` doesn't call `LibUsb.claimInterface`. The session manager in §3.5 does this for you.
- **The device re-enumerates after `switchDeviceToAoa`.** The original `Device` object becomes invalid, so scan again for VID `0x18D1`. The session manager in §3.5 does this for you.
- **A closed session manager can't restart.** After `close()`, create a new one. Use `stopPolling()` if you plan to start again.
- **Filter strings must match.** If `manufacturer`/`model` differ from `accessory_filter.xml`, the app won't launch automatically.
- `sendFile` **and** `sendDirectory` **have no acknowledgement.** `true` means the bytes were written to USB, not that the phone saved them. See §3.12 for how to check.
- **Responses must match the contract in §5 exactly.** A missing reply leaves the host waiting in `receiveSecure()`.
- **Keys are ephemeral.** A new ECDH key pair is created for every `performHandshake`, and there is no persistent pairing or identity check. Treat the physical cable as the trust boundary.

9. Troubleshooting

Symptom	Likely cause / fix
`switchDeviceToAoa` returns `false`	No permission to open the device (Linux udev / Windows driver), or the device doesn't support AOA
`LibUsb.open` → `ERROR_ACCESS (-3)` on Windows	`adb` server holds the device. Run `adb kill-server`
`LibUsb.open` → `ERROR_NOT_SUPPORTED (-12)`	WinUSB driver missing for `18D1:2D00/2D01`. Run the driver installer, then retry after about 1–2 s
`claimInterface` → `ERROR_BUSY`	Another process or kernel driver owns it. Detach the kernel driver or close the other app
`performHandshake` returns `false` after ~30 s	Android app didn't open the accessory or isn't running the handshake as responder
`createAndroidAccessoryTransport` returns `null`	No USB permission, or the accessory is already opened by another component
App doesn't open on plug-in	`accessory_filter.xml` strings don't match `switchDeviceToAoa` arguments
Transfer stalls on Android	Write sizes are already split into 16 KB bursts by `AndroidAoaTransport`. Check that the receiver loop is consuming packets
`listRemoteFiles` returns empty	Android didn't reply with the `[4: count]` header first, or has no storage permission

Project layout

```
secureqt-sdk/src
├── commonMain/…/sdk
│   ├── SecureQtSdk.kt              Public facade
│   ├── channel/SecureChannel.kt    Encrypted, framed channel
│   ├── client/                     ISecureTransferClient + impl (host operations)
│   ├── crypto/                     ECDH P-256 + AES-256-GCM
│   ├── protocol/                   PacketCodec, PacketTypes, StandardCommandIds, Builder/Parser
│   ├── buffer/ model/ util/ logging/
│   └── transport/IUsbTransport.kt
├── jvmMain/…/sdk                   DesktopAoaTransport, DesktopAoaConnector, desktop extensions
│   └── host/                       Session manager, sessions, device discovery, connect flow
└── androidMain/…/sdk               AndroidAoaTransport, Android extensions
```

See also, in the repo root: `SECUREQT_SDK_PROTOCOL_SPECIFICATION.md`, `Protocol.md`, `Architecture_and_Security.md`.